MySQL Assignment 1
Indonesian Traditional Dance

3461260 , *Student of, University of Stirling,*

I. Scenario Design

THE Indonesian Traditional Dance database is designed to comprehensively document and manage information about Indonesia's rich cultural heritage of traditional dances. The system serves as a centralized repository for storing detailed information about the various aspects of these dances, including their origins, the artists who perform them, and the performances staged across different regions.

The database captures several key entities that form the core of this cultural management system:

The regions represent the geographical areas of Indonesia where traditional dances originate and are performed. Each region can be associated with multiple dances, reflecting the cultural diversity in Indonesia's provinces and islands. Regional information helps preserve the cultural context and authenticity of each dance form.

Dances are the primary focus of the database, containing details about specific traditional dance forms such as their names, characteristics, historical significance, and cultural background. Each dance can be linked to one or more regions, as many dances have deep roots in particular areas of Indonesia.

Dancers are the artists and performers who keep these traditions alive through their performances. The database records information about individual dancers, their expertise, specializations, and performance history.

Performances document actual dance events and presentations. Each performance records when and where a traditional dance was performed, who participated in it, and other relevant event details. Performances serve as the connection point linking specific dances with the dancers who perform them and the regions they represent.

This scenario design establishes the foundational structure needed to preserve, track, and celebrate Indonesian traditional dance as a vital component of the nation's cultural identity.

II. ER Diagram

The Entity-Relationship (ER) Diagram illustrates the structure of the Indonesian Traditional Dance database as it progresses through the normalization process. Normalization is a systematic approach to organizing data to eliminate redundancy, reduce data anomalies, and improve database efficiency and integrity.

M. Shell was with the Department of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA, 30332 USA e-mail: (see http://www.michaelshell.org/contact.html).

J. Doe and J. Doe are with Anonymous University.

Manuscript received April 19, 2005; revised August 26, 2015.



Fig. 1. Zero Normal Form ER Diagram

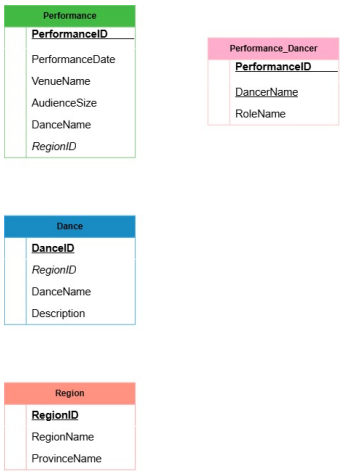


Fig. 2. First Normal Form ER Diagram

A. Zero Normal Form (0NF)

In the 0NF stage, the database exists in its most unstructured form. The ER diagram shows a single, monolithic structure with overlapping entities. The main entities—Performance, Dance, Region, and Dancer—are interconnected without clear separation of concerns. At this stage, the database may contain repeating groups within single records. For example, a Performance record might directly contain multiple Dancer entries and Region information embedded within it, causing significant data redundancy.



Fig. 3. Second Normal Form ER Diagram



Fig. 4. Third Normal Form ER Diagram

B. First Normal Form (1NF)

Transitioning to $1NF$ requires eliminating repeating groups. The ER diagram now shows a clearer separation of entities. Individual tables are created for Dance, Region, Dancer, and Performance, each containing atomic (indivisible) values only. A $Performance_Dancer$ junction table is introduced to handle the many-to-many relationship between performances and dancers, since one performance can involve multiple dancers and one dancer can perform in multiple performances. This eliminates repeating groups and ensures each attribute contains only single values.

C. Second Normal Form (2NF)

The progression to 2NF addresses partial dependencies by ensuring all non-key attributes are fully dependent on the entire primary key. The ER diagram maintains the entities from 1NF but refines the relationships. The $Performance_Dancer$ table now has a composite primary key consisting of both $PerformanceID$ and $DancerID$, ensuring that dancer information is not partially dependent on just one part of the key. Each entity becomes more focused on a single subject matter.

D. Third Normal Form (3NF)

In the final 3NF stage, the database achieves its most normalized state by eliminating transitive dependencies. The ER diagram shows clean, independent entities where Region stands as a separate table, Performance is independent with only its own attributes, Dance maintains its own identity, and $Dancer$ is self-contained. The $Performance_Dancer$ junction table clearly represents only the relationship between performances and dancers without introducing any transitive dependencies. Each table now represents a distinct entity with no non-key attribute depending on another non-key attribute.

Through this normalization process, the Indonesian Traditional Dance database evolves from a redundant, unstructured format into a well-organized, efficient relational structure that maintains data integrity while minimizing storage requirements and preventing update anomalies.

Region ID	Region Name	Province Name
1	Java	East Java
2	Java	Central Java
3	Bali	Bali
4	Sumatera	North Sumatera
5	Borneo(Kalimantan)	South Kalimantan
6	Sulawesi	South Sulawesi
7	Lombok	West Nusa Tenggara
8	Papua	Central Papua
9	Aceh	Aceh
10	Maluku	Maluku

Fig. 5. Region Table

III. Sample Data

The database includes sample data to demonstrate the relationships between the tables. The sample data is organized into five tables: Region, Dance, Dancer, Performance, and PerformanceRole. Each table contains example records that represent Indonesian traditional dance information.

1) Region Table Sample Data: The Region table stores regional information such as region ID, region name, and province name.

2) Dance Table Sample Data: The Dance table contains information about traditional dances and their related regions.

3) Dancer Table Sample Data: The Dancer table stores dancer identity and birth information.

Dance ID	Region ID	DanceName	Description
101	2	Ramayana Ballet	Best dance-drama from Ramayana story, performed with intricate hand movement and elaborate costume
102	1	Reog Ponorogo	It characterised about a large-like mask, mixing martial arts with cultural storytelling
103	3	Legong	Beautiful Balinese classical dance performed by young girls, familiar with finger movement and swaying hips
104	2	Serimpi	Ancient Court dance from Central Java affecting four female dancer in a slow, reflective performance
105	4	Poco-Poco	Modern social dance from North Sumatera decribe about synchronised group movement
106	5	Tari Kancet	Traditional dance from Borneo representing dayak culture, nature, and ritual symbolism
107	6	Pakarena	Graceful Sulawesi dance expressing courtesy, harmony, and feminine movement
108	7	Gendang Beleq	Energetic Lombok performance featuring drums and strong rhythmic movement
109	8	Yospan	Papuan social dance combining joy, friendship, and group coordination
110	9	Seudati	Acehnese dance with energetic steps, chanting, and strong cultural expression

Fig. 6. Dance Table

Dancer ID	Region ID	Dancer Name	Date of Birth
201	2	Siti Nurmaiza	1991-05-15
202	3	Ni Made Ayu	1997-08-22
203	1	Budi Santoso	1987-03-10
204	2	Ratna Dewi	1994-11-30
205	4	Enang Suryadi	1989-07-14
206	3	I Gusti Made	1996-01-20
207	2	Dewi Lestari	1995-05-05
208	5	Andi Hidayat	1992-09-18
209	6	Maya Sari	1998-02-11
210	7	Rizky Pratama	1993-12-27

Fig. 7. Dancer Table

Performance ID	Dance ID	Performance date	Venue Name	Audience Size
301	101	2026-11-14	Prambanan Temple Amphitheater	2500
302	103	2026-11-17	Bali Arts Festival Grand Stage	1800
303	102	2026-11-19	Ponorogo Cultural Center	1200
304	104	2026-11-21	Yogyakarta Palace (Keraton)	950
305	105	2026-11-24	Medan Convention Center	3200
306	101	2026-11-30	National Museum Jakarta	1500
307	103	2026-12-04	Bali Arts Festival Grand Stage	2100
308	106	2026-12-08	Dayak Cultural Hall	1100
309	107	2026-12-12	Makassar Art Center	1400
310	108	2026-12-16	Lombok Heritage Stage	1600

Fig. 8. Performance Table

4) Performance Table Sample Data: The Performance table records dance performance events, including date, venue, and audience size.

5) PerformanceRole Table Sample Data: The Performance-Role table links dancers to performances and the roles they played.

IV. SQL Table Creation

This section defines the database schema derived from the ERD. Each table uses data types suited to its attributes, and primary and foreign keys enforce referential integrity between entities.

A. Create Tables

```
CREATE DATABASE IF NOT EXISTS indonesian_dance;
USE indonesian_dance;

-- Region: stores regions and provinces
CREATE TABLE Region (
    RegionID       INT           NOT NULL AUTO_INCREMENT,
    RegionName     VARCHAR(50)   NOT NULL,
    ProvinceName   VARCHAR(100)  NOT NULL,
    CONSTRAINT pk_region PRIMARY KEY (RegionID)
);

-- Dance: traditional dances linked to a region
CREATE TABLE Dance (
    DanceID        INT           NOT NULL AUTO_INCREMENT,
    RegionID       INT           NOT NULL,
    DanceName      VARCHAR(100)  NOT NULL,
    Description    TEXT,
    CONSTRAINT pk_dance PRIMARY KEY (DanceID),
    CONSTRAINT fk_dance_region FOREIGN KEY (RegionID)
        REFERENCES Region (RegionID)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

-- Dancer: performers linked to a region
CREATE TABLE Dancer (
    DancerID       INT           NOT NULL AUTO_INCREMENT,
    RegionID       INT           NOT NULL,
    DancerName     VARCHAR(100)  NOT NULL,
    DateOfBirth    DATE,
    CONSTRAINT pk_dancer PRIMARY KEY (DancerID),
    CONSTRAINT fk_dancer_region FOREIGN KEY (RegionID)
        REFERENCES Region (RegionID)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

-- Performance: scheduled performances of a dance
CREATE TABLE Performance (
    PerformanceID     INT           NOT NULL AUTO_INCREMENT,
    DanceID           INT           NOT NULL,
    PerformanceDate   DATE          NOT NULL,
    VenueName         VARCHAR(150)  NOT NULL,
    AudienceSize      INT           NOT NULL DEFAULT 0,
    CONSTRAINT pk_performance PRIMARY KEY (PerformanceID),
    CONSTRAINT fk_performance_dance FOREIGN KEY (DanceID)
        REFERENCES Dance (DanceID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_audience_nonnegative CHECK (AudienceSize >= 0)
);

-- PerformanceRole: associative table linking dancers to performances
CREATE TABLE PerformanceRole (
    PerformanceID  INT           NOT NULL,
    DancerID       INT           NOT NULL,
    RoleName       VARCHAR(50)   NOT NULL,
    CONSTRAINT pk_performancerole PRIMARY KEY (PerformanceID, DancerID),
    CONSTRAINT fk_pr_performance FOREIGN KEY (PerformanceID)
        REFERENCES Performance (PerformanceID)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_pr_dancer FOREIGN KEY (DancerID)
        REFERENCES Dancer (DancerID)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

Performance ID	Dancer ID	Role Name
301	201	Lead Rama
301	204	Lead Sita
301	207	Supporting Dancer
302	202	Lead Legong Dancer
302	206	Supporting Legong Dancer
303	203	Lead Warrior
303	205	Supporting Warrior
304	204	Lead Performer
304	207	Supporting Dancer
305	205	Choreographer & Lead
306	201	Lead Rama
306	204	Lead Sita
307	202	Lead Legong Dancer
307	206	Supporting Legong Dancer
308	208	Lead Dayak Performer
308	205	Supporting Dayak Performer
309	209	Lead Pakarena Dancer
309	210	Supporting Performer
310	210	Lead Gendang Performer
310	208	Supporting Drummer

Fig. 9. PerformanceRole Table

B. Load Sample Data with LOAD DATA INFILE

The CSV files from Step 3 are loaded in dependency order so foreign keys are satisfied. Parent tables load first.

```
SET FOREIGN_KEY_CHECKS = 0;

LOAD DATA INFILE '/var/lib/mysql-files/region.csv'
INTO TABLE Region
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(RegionID, RegionName, ProvinceName);

LOAD DATA INFILE '/var/lib/mysql-files/dance.csv'
INTO TABLE Dance
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(DanceID, RegionID, DanceName, Description);

LOAD DATA INFILE '/var/lib/mysql-files/dancer.csv'
INTO TABLE Dancer
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(DancerID, RegionID, DancerName, DateOfBirth);

LOAD DATA INFILE '/var/lib/mysql-files/performance.csv'
INTO TABLE Performance
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(PerformanceID, DanceID, PerformanceDate, VenueName, AudienceSize);

LOAD DATA INFILE '/var/lib/mysql-files/performancerole.csv'
INTO TABLE PerformanceRole
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(PerformanceID, DancerID, RoleName);
```

```
SET FOREIGN_KEY_CHECKS = 1;
```

V. SQL Queries

This section demonstrates the database functionality using the schema defined in the Table Creation section (Region, Dance, Dancer, Performance, PerformanceRole).

A. Query 1: Using LIKE on an alphanumeric field

```
SELECT DancerName, DateOfBirth
FROM Dancer
WHERE DancerName LIKE 'Siti%';
```

Purpose: Finds dancers whose names start with a given prefix. This supports search features in the dance registry, such as an organizer looking up a performer by partial name without needing the exact spelling.

B. Query 2: JOIN across multiple entities

```
SELECT p.PerformanceDate,
       p.VenueName,
       d.DanceName,
       r.RegionName,
       dr.DancerName,
       pr.RoleName
FROM Performance p
JOIN Dance d          ON p.DanceID = d.DanceID
JOIN Region r         ON d.RegionID = r.RegionID
JOIN PerformanceRole pr ON p.PerformanceID = pr.PerformanceID
JOIN Dancer dr        ON pr.DancerID = dr.DancerID
ORDER BY p.PerformanceDate, dr.DancerName;
```

Purpose: Produces a full performance roster showing which dancer played which role, in which dance, from which region, at which venue and date. This is the main report a production manager would use to plan a performance schedule.

C. Query 3: ¿ operator on a single entity

```
SELECT PerformanceID, VenueName, AudienceSize
FROM Performance
WHERE AudienceSize > 1000;
```

Purpose: Identifies large-audience performances (over 1,000 attendees). Event organizers can use this to prioritize venues, assign more staff, or measure which shows draw the biggest crowds.

D. Query 4: GROUP BY with aggregation

```
SELECT d.DanceName,
       COUNT(p.PerformanceID)  AS TotalPerformances,
       SUM(p.AudienceSize)     AS TotalAudience,
       AVG(p.AudienceSize)     AS AvgAudience
FROM Performance p
JOIN Dance d ON p.DanceID = d.DanceID
GROUP BY d.DanceName
ORDER BY TotalAudience DESC;
```

Purpose: Summarizes how many times each dance is performed, its total audience reach, and its average audience size. This helps the organization compare dance popularity and plan future programming.

E. Query 5: IN operator on a single entity

```
SELECT DancerID, DancerName, RegionID
FROM Dancer
WHERE RegionID IN (1, 2, 3);
```

Purpose: Retrieves dancers from a selected set of regions (here, Java and Bali). This is useful when a tour or festival targets specific regions and the coordinator needs only those performers.

F. Query 6: UPDATE multiple rows

```
UPDATE Performance
SET AudienceSize = AudienceSize + 100
WHERE PerformanceDate BETWEEN '2026-11-14' AND '2026-11-21';
```

Purpose: Corrects or adjusts audience figures for several performances in the same period, for example after a late ticket count or an added seating section. Because the update affects multiple rows in one statement, it shows how bulk maintenance can be done efficiently.

Discussion of Complexity

One part of the scenario that is difficult to implement purely in SQL is the assignment of dancers to roles within a performance, particularly the rules that govern whether a dancer is eligible for a given role. The ‘PerformanceRole‘ table records that dancer X played role Y in performance Z, but it does not enforce the business rules behind that assignment. For example, a lead role may require a dancer with a minimum number of past performances, a supporting role may require a dancer from the same region as the dance, and a choreographer role may need to be held by someone who has not also performed as a lead in the same show. Expressing these rules in SQL means writing correlated subqueries, multi-table joins, and aggregation checks, and then embedding them in ‘CHECK‘ constraints or triggers. MySQL restricts subqueries in ‘CHECK‘ constraints and has historically had limited support for them, so most of this logic would have to live in ‘BEFORE INSERT‘ or ‘BEFORE UPDATE‘ triggers. Those triggers are harder to write, test, and maintain, and their behaviour is less transparent to other developers than a simple foreign key.

The difficulty grows when the rules depend on information outside the database or on judgement. Whether a dancer is ”qualified” for a lead role may depend on training certificates, physical condition, audition scores, or the choreographer’s artistic decisions, none of which are stored in the schema. Even if such data were modelled, deciding the eligibility rule often involves weighting several factors, applying exceptions, and changing criteria from season to season. SQL is well suited to set-based retrieval and enforcing simple integrity, but it is poorly suited to multi-step decision logic that changes over time. Procedural code in an application layer, or a rules engine, would usually be a better place for this logic, with the database keeping only the data and basic constraints.

A related challenge is scheduling conflicts. Checking that the same dancer is not assigned to two performances at overlapping times, or that a venue is not double-booked, requires comparing time intervals across rows in the same table. In

SQL this is typically done with a self-join or an ‘EXISTS‘ subquery on the ‘PerformanceRole‘ and ‘Performance‘ tables, which works for small datasets but becomes costly as the number of performances grows. Without an explicit start time and end time for each performance, the check becomes even less reliable, because the schema currently stores only a date. Handling this properly would require a schema change, careful indexing, and possibly exclusion constraints (available in PostgreSQL but not in MySQL), which shows how a seemingly simple business requirement can push the limits of relational SQL.